Bahria University

Lahore Campus

Department of Computer Sciences

Information Security

Assignment 4 – Practical Tasks

CLO 3: Identify appropriate techniques to tackle and solve problems in the discipline of information security.

Student Name	Muhammad Ahsan Ashfaq
Roll No	03-134231-036
Program / Semester	BSCS 8 (A/B)
Instructor	Dr. Nadeem Sarwar
Assignment Date	02-11-2026
Submission Date	08-11-2026
Max Marks	5

Overview

This report covers the two practical tasks of Assignment 4. Only free and open-source tools were used. All testing was done on my own laptop against a deliberately vulnerable test application.

Task	Tool	Target / Environment
Task 1 – Web application vulnerability scan	OWASP ZAP 2.x (Docker image zaproxy/zap-stable)	OWASP Juice Shop at http://localhost:3000 (Docker)
Task 2 – Basic firewall rules	Windows Defender Firewall with Advanced Security (PowerShell NetSecurity cmdlets)	Local Windows 11 laptop

Task 1: Scanning a Vulnerable Web Application

Objective: identify and explain at least three vulnerabilities in a test web application using OWASP ZAP.

1.1 Environment setup

OWASP Juice Shop is an intentionally insecure web shop maintained by OWASP for security training. It was started locally with Docker:

```
docker pull bkimminich/juice-shop
docker run -d --name juice-shop -p 3000:3000 bkimminich/juice-shop
```

The application was then opened in the browser at http://localhost:3000 to confirm it was running.

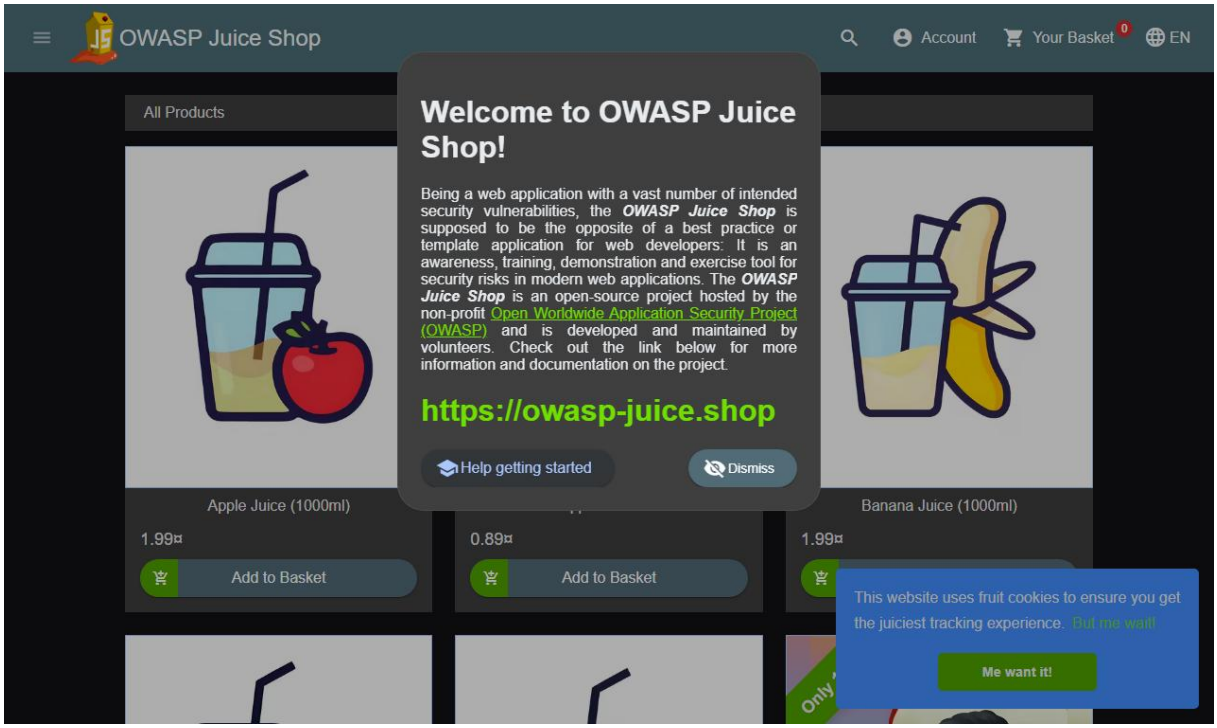


Figure 1: OWASP Juice Shop running locally on port 3000

1.2 Scanning with OWASP ZAP

OWASP ZAP (Zed Attack Proxy) was run from its official Docker image using the full-scan script. The full scan:

- Spiders the application to discover pages and API endpoints,
- Runs the passive scanner on every response (headers, cookies, information leaks),
- Runs the active scanner, which sends attack payloads (SQL injection, XSS, path traversal, etc.),
- Writes HTML, JSON and Markdown reports.

```
docker pull zaproxy/zap-stable
docker run --rm -v "%cd%:/zap/wrk" zaproxy/zap-stable zap-full-scan.py ^
    -t http://host.docker.internal:3000 -r zap_report.html -J zap_report.json -m 3 -T
25
```

Options: -t is the target URL (host.docker.internal is how the ZAP container reaches the host), -m 3 sets 3 minutes of spidering, -T 25 sets a 25-minute maximum scan time, and -r/-J set the report files.

1.3 Scan results summary

ZAP raised 14 distinct alert types: 0 High, 6 Medium, 4 Low, 4 Informational.

Risk	Alert	Instances	CWE
Medium	Backup File Disclosure	31	530

Medium	Bypassing 403	6	348
Medium	CORS Misconfiguration	5	942
Medium	Content Security Policy (CSP) Header Not Set	5	693
Medium	Cross-Domain Misconfiguration	5	264
Medium	HTTP Only Site	1	311
Low	Cross-Origin-Embedder-Policy Header Missing or Invalid	5	693
Low	Cross-Origin-Opener-Policy Header Missing or Invalid	5	693
Low	Dangerous JS Functions	2	749
Low	Deprecated Feature Policy Header Set	5	16
Informational	Modern Web Application	5	-1
Informational	Storable and Cacheable Content	1	524
Informational	Storable but Non-Cacheable Content	5	524
Informational	User Agent Fuzzer	5	0



Sites: https://host.docker.internal:3000 http://host.docker.internal:3000

Generated on Sun, 8 Nov 2026 17:58:25

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	6
Low	4
Informational	4
False Positives:	0

Figure 2: OWASP ZAP report – summary of alerts by risk

Alerts

Name	Risk Level	Number of Instances
Backup File Disclosure	Medium	31
Bypassing 403	Medium	6
CORS Misconfiguration	Medium	Systemic
Content Security Policy (CSP) Header Not Set	Medium	Systemic
Cross-Domain Misconfiguration	Medium	Systemic
HTTP Only Site	Medium	1
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	Systemic
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	Systemic
Dangerous JS Functions	Low	2
Deprecated Feature Policy Header Set	Low	Systemic
Modern Web Application	Informational	Systemic
Storable and Cacheable Content	Informational	1
Storable but Non-Cacheable Content	Informational	Systemic
User Agent Fuzzer	Informational	Systemic

Figure 3: OWASP ZAP report – list of alerts found

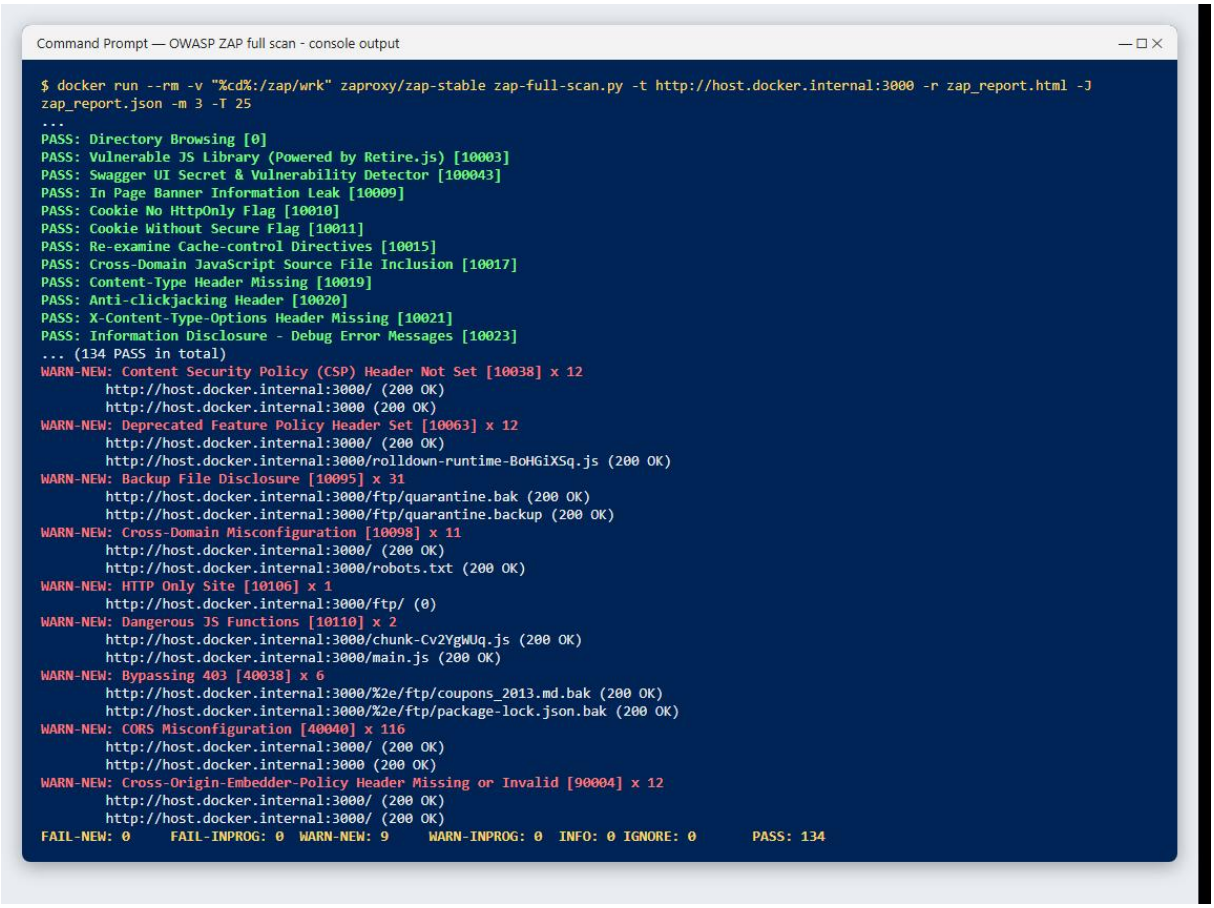


Figure 4: ZAP scan console output (PASS / WARN results)

Alert Detail

Medium	Backup File Disclosure
Description	A backup of the file was disclosed by the web server.
URL	http://host.docker.internal:3000/ftp/quarantine%20-%20Copy
Node Name	http://host.docker.internal:3000/ftp/quarantine - Copy
Method	GET
Parameter	
Attack	http://host.docker.internal:3000/ftp/quarantine%20-%20Copy
Evidence	
Other Info	A backup of [http://host.docker.internal:3000/ftp/quarantine] is available at [http://host.docker.internal:3000/ftp/quarantine%20-%20Copy]
URL	http://host.docker.internal:3000/ftp/quarantine%20-%20Copy%20(2)
Node Name	http://host.docker.internal:3000/ftp/quarantine - Copy (2)
Method	GET
Parameter	
Attack	http://host.docker.internal:3000/ftp/quarantine%20-%20Copy%20(2)

Figure 5: ZAP alert detail – Backup File Disclosure instances

1.4 Manual verification

Scanners can report false positives, so each finding was checked by hand with curl. This confirmed the backup-file exposure (the backup was downloaded with a poison null byte %2500.md), the downloadable KeePass password database, the wildcard CORS header, and

the missing CSP/HSTS headers. One alert, "Bypassing 403", turned out to be a false positive: the /%2e/ftp/... URL only returns the application's index page, not the protected file. It is therefore not counted as a vulnerability.

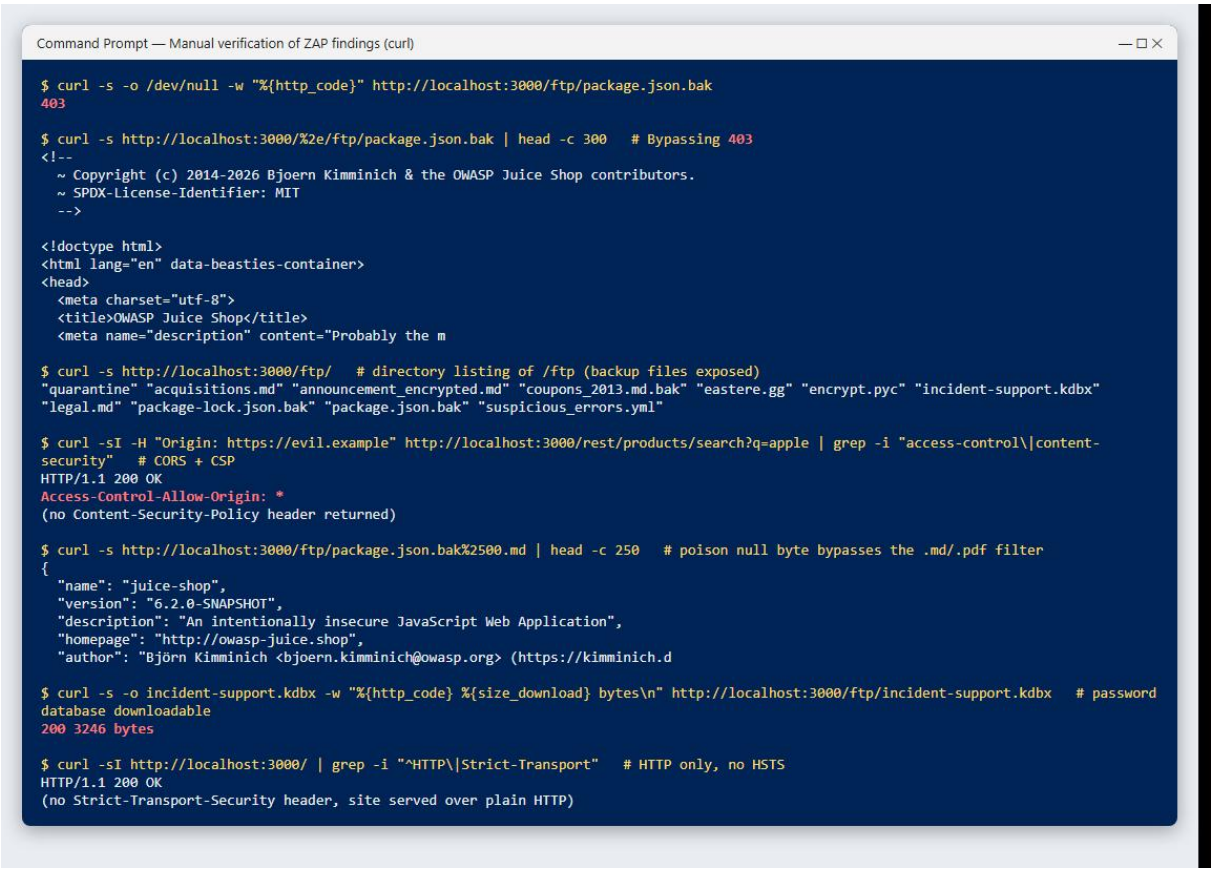


Figure 6: Manual verification of the ZAP findings with curl

1.5 Vulnerabilities found and mitigations

Vulnerability 1: Backup File Disclosure

Risk level	Medium (confidence: 2)
CWE / WASC	CWE-530 / WASC-34
Example URL	http://host.docker.internal:3000/ftp/quarantine%20-%20Copy
Method / parameter	GET
Evidence	http://host.docker.internal:3000/ftp/quarantine%20-%20Copy
Instances found	31

Description: The /ftp directory of the web server is publicly browsable and contains backup and sensitive files such as package.json.bak, coupons_2013.md.bak, package-lock.json.bak, suspicious_errors.yml and incident-support.kdbx (a KeePass password database). ZAP found 31 instances of backup copies. The server only tries to restrict downloads by file extension

(.md/.pdf), and this check is bypassed with a poison null byte (package.json.bak%2500.md). Manual testing downloaded the backup file and the password database.

Impact: Backups reveal source code, dependency versions (useful for finding known CVEs), coupon codes, internal configuration and credentials. The KeePass database could be cracked offline. This maps to OWASP Top 10 A01 Broken Access Control and A05 Security Misconfiguration.

Mitigation:

- Never store backups, archives or secret files inside the web root; keep them in a separate, access-controlled location.
- Disable directory listing (e.g. remove serve-index in Express; Options -Indexes in Apache; autoindex off in Nginx).
- Use an allow-list of files that may be served and enforce authorization on every request.
- Normalise and validate file names (reject %00, %2500 and path traversal sequences) instead of relying on extension checks.
- Add a CI check or deployment rule that blocks *.bak, *.old, *.zip and *.kdbx files from being deployed.

Vulnerability 2: CORS Misconfiguration

Risk level	Medium (confidence: 3)
CWE / WASC	CWE-942 / WASC-14
Example URL	http://host.docker.internal:3000
Method / parameter	GET
Evidence	origin: http://pCZROLtt.com
Instances found	5

Description: The server returns the header Access-Control-Allow-Origin: * for every response, including the REST API (/rest/products/search). Any website on the Internet is therefore allowed by the browser to read responses from the Juice Shop API.

Impact: A malicious website visited by a user can call the API from the victim's browser and read the data returned. Combined with token-based authentication stored in the browser, it can be used to steal personal data (OWASP A05 Security Misconfiguration, CWE-942).

Mitigation:

- Replace the wildcard with an explicit allow-list of trusted origins (e.g. https://shop.example.com).
- Validate the Origin header on the server and never reflect arbitrary origins back.
- Do not combine Access-Control-Allow-Credentials: true with broad origins.
- Only enable CORS on the endpoints that actually need cross-origin access.

Vulnerability 3: Content Security Policy (CSP) Header Not Set

Risk level	Medium (confidence: 3)
CWE / WASC	CWE-693 / WASC-15
Example URL	http://host.docker.internal:3000
Method / parameter	GET
Evidence	-
Instances found	5

Description: Responses do not include a Content-Security-Policy header. CSP tells the browser which sources of scripts, styles, images and frames are allowed. Without it the browser will run any injected script.

Impact: If an attacker finds a Cross-Site Scripting (XSS) or HTML-injection bug (Juice Shop has several), nothing stops the injected JavaScript from running and stealing session tokens, logging keystrokes or defacing the page. The site can also be framed by other sites (clickjacking) (CWE-693).

Mitigation:

- Add a strict CSP header, for example: Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'; base-uri 'self'.
- Use nonces or hashes instead of 'unsafe-inline' for any required inline scripts.
- Start with Content-Security-Policy-Report-Only to find violations, then enforce it.
- In Node/Express the helmet middleware (helmet.contentSecurityPolicy) adds it easily. CSP is defence in depth, so input/output encoding is still needed to fix XSS.

Vulnerability 4: HTTP Only Site

Risk level	Medium (confidence: 2)
CWE / WASC	CWE-311 / WASC-4
Example URL	http://host.docker.internal:3000/ftp/
Method / parameter	GET
Evidence	-
Instances found	1

Description: The application is served only over plain HTTP. There is no HTTPS endpoint and no Strict-Transport-Security (HSTS) header, so all traffic, including login credentials and session tokens, is sent unencrypted.

Impact: Anyone on the same network (public Wi-Fi, a compromised router, ISP) can sniff passwords and tokens or modify pages in transit (man-in-the-middle). This maps to OWASP A02 Cryptographic Failures (CWE-311).

Mitigation:

- Serve the site only over HTTPS using a TLS certificate (e.g. free Let's Encrypt) with TLS 1.2+.
- Redirect all HTTP requests to HTTPS (301).
- Send Strict-Transport-Security: max-age=31536000; includeSubDomains so browsers always use HTTPS.
- Mark cookies as Secure and HttpOnly.

Task 2: Configuring Basic Firewall Rules (Windows Defender Firewall)

Objective: demonstrate network traffic control with simple firewall rules. Tool: Windows Defender Firewall with Advanced Security, configured with PowerShell (NetSecurity module) in an Administrator PowerShell window. The same settings can be made in the GUI (wf.msc).

2.1 Rules configured

#	Purpose	PowerShell command
1	Block incoming traffic on TCP port 23 (Telnet)	New-NetFirewallRule -DisplayName "A4 Rule1 - Block Inbound Telnet (TCP 23)" -Direction Inbound -Protocol TCP -LocalPort 23 -Action Block -Profile Any
2	Allow localhost (127.0.0.1) access to TCP port 8080	New-NetFirewallRule -DisplayName "A4 Rule2 - Allow Localhost to TCP 8080" -Direction Inbound -Protocol TCP -LocalPort 8080 -RemoteAddress 127.0.0.1 -Action Allow -Profile Any
3	Log dropped incoming packets	Set-NetFirewallProfile -Profile Domain,Private,Public -LogBlocked True -LogFileName "%SystemRoot%\System32\LogFiles\Firewall\pfirewall.log" -LogMaxSizeKilobytes 4096

Equivalent GUI settings (wf.msc)

1. Rule 1: Inbound Rules → New Rule → Port → TCP, Specific local port 23 → Block the connection → Domain/Private/Public → Name "A4 Rule1 - Block Inbound Telnet (TCP 23)".
2. Rule 2: Inbound Rules → New Rule → Custom → All programs → Protocol TCP, Local port 8080 → Scope: Remote IP address 127.0.0.1 → Allow the connection → all profiles → Name "A4 Rule2 - Allow Localhost to TCP 8080".
3. Rule 3: Windows Defender Firewall Properties → for each profile (Domain, Private, Public) → Logging → Customize → Log dropped packets: Yes, file %SystemRoot%\System32\LogFiles\Firewall\pfirewall.log, size 4096 KB.

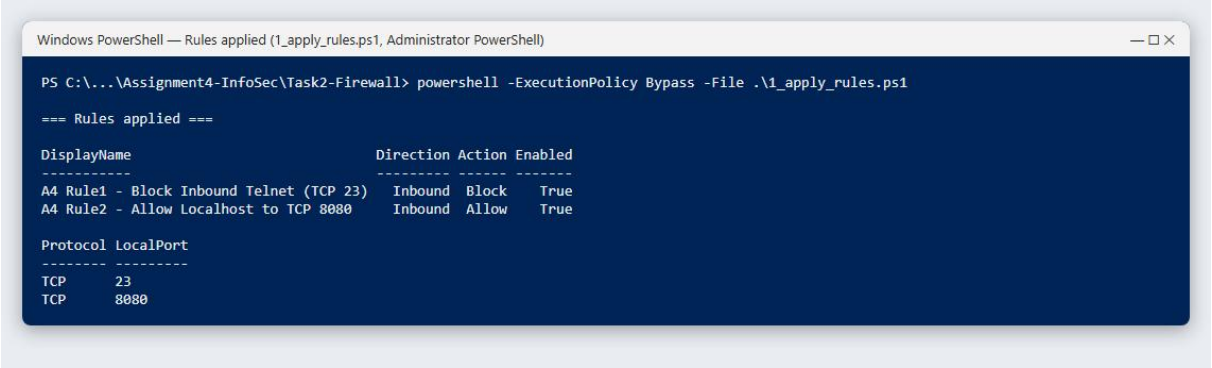


Figure 7: Rules created by 1_apply_rules.ps1 (Administrator PowerShell)

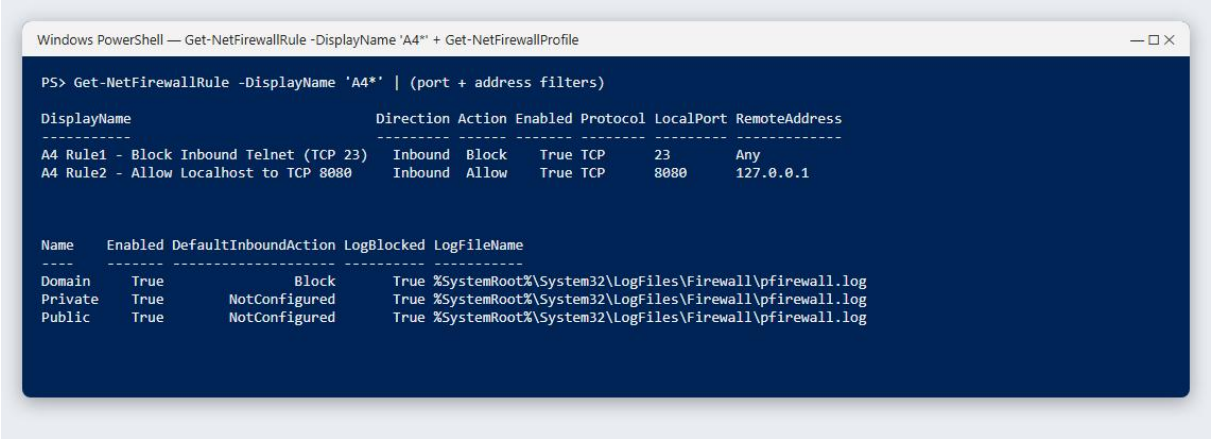


Figure 8: Rule details (ports, remote address) and logging enabled on all profiles

2.2 Testing the rules

Test of Rule 2 – localhost access to port 8080

A temporary TCP listener was started on 127.0.0.1:8080 and Test-NetConnection was used to connect from the same machine. Result: TcpTestSucceeded = True, so local access to port 8080 is allowed.

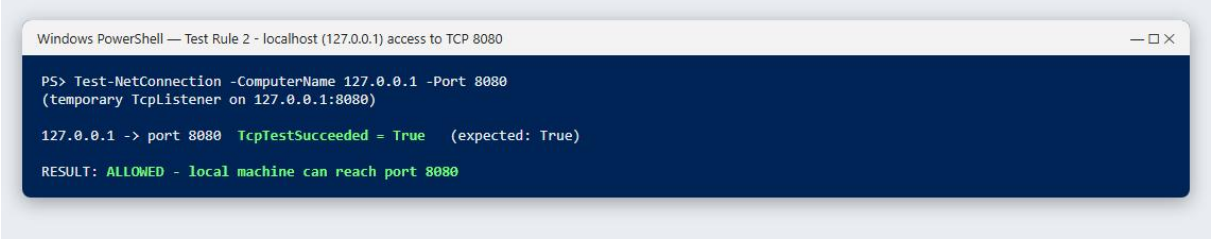


Figure 9: Rule 2 test – localhost can reach port 8080

Test of Rule 1 – Telnet port 23 from another host

Windows does not filter traffic that a machine sends to itself, so a separate host was needed. The WSL2 virtual machine (IP 172.30.147.71) was used as the remote client. Its traffic reaches Windows through the vEthernet (WSL) adapter, where the Windows/Hyper-V firewall applies. A Python 3.12 listener was started on port 23 and on port 2323 as a control. This Python has an inbound Allow rule, so the only difference between the two ports is Rule 1. The netcat (nc) tool in the VM was then used like a telnet client.

Connection from WSL VM	Result	Meaning
172.30.144.1 port 23	Operation timed out	BLOCKED by Rule 1 (packet dropped)
172.30.144.1 port 2323	open / connected	Allowed (control – no block rule)
127.0.0.1 port 23 (local)	listening = True	The service was running, so the timeout is caused by the firewall

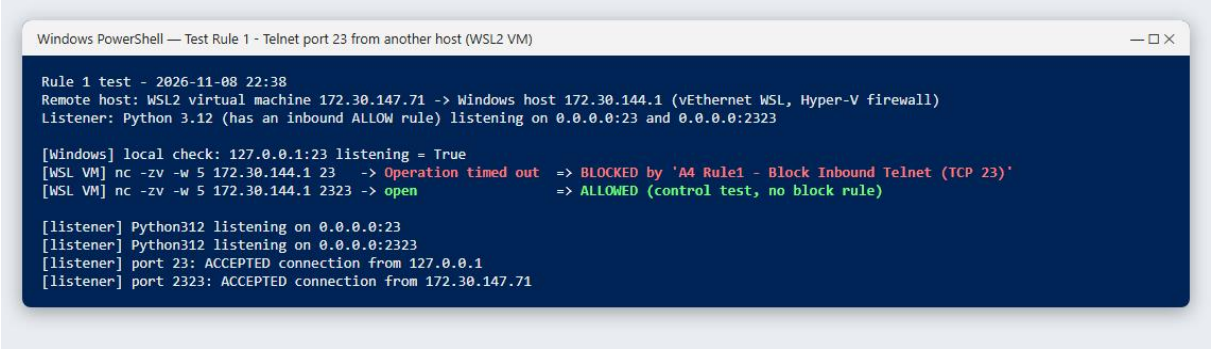


Figure 10: Rule 1 test – port 23 blocked from another host, port 2323 allowed

Test of Rule 3 – logging of dropped packets

After logging was enabled, pfirewall.log began recording every dropped inbound packet. Each line has the date, time, action (DROP), protocol, source IP, destination IP, source port, destination port and size. The entries below show multicast DNS (UDP 5353) packets from other devices on the Wi-Fi network being dropped and logged.

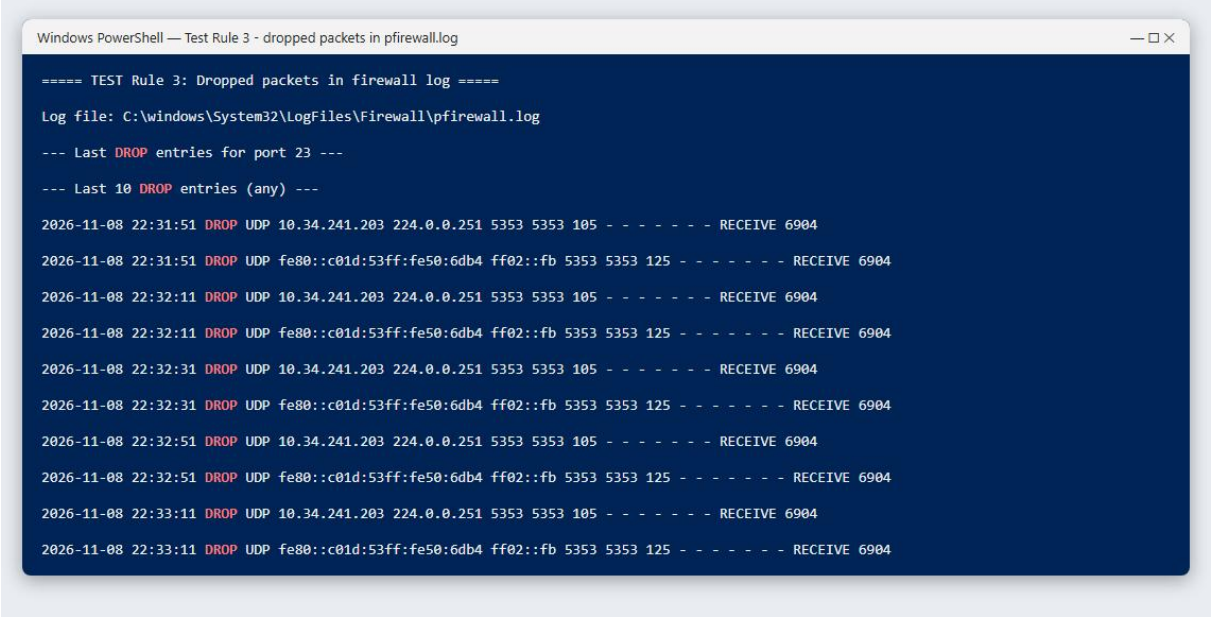


Figure 11: Rule 3 test – DROP entries recorded in pfirewall.log

2.3 Explanation of each rule

Rule	What it does	Why it is useful
1. Block TCP 23	Drops every inbound connection attempt to port 23 on all network profiles.	Telnet sends usernames, passwords and commands in plain text and is a common target of brute-force attacks and botnets (e.g. Mirai). Blocking it reduces the attack surface; SSH (port 22) should be used instead.

2. Allow 127.0.0.1 → TCP 8080	Allows connections to port 8080 only when the source address is the local machine.	Developers often run test servers or admin panels on 8080. Limiting them to localhost lets the developer use them while devices on the network cannot reach them (least-privilege / need-to-know access).
3. Log dropped packets	Writes a line to pfirewall.log for every packet the firewall drops.	Gives visibility for troubleshooting and incident response: it shows port scans, repeated connection attempts and which rule is stopping traffic. Logs can be sent to a SIEM for alerting and audit.

2.4 Removing the rules

```
Remove-NetFirewallRule -DisplayName "A4*"
Set-NetFirewallProfile -Profile Domain,Private,Public -LogBlocked False
```

Conclusion

In Task 1, OWASP ZAP found several real weaknesses in OWASP Juice Shop. The most important are described above with their risk and mitigation, and they map to the OWASP Top 10: injection, security misconfiguration and broken access control. In Task 2, three Windows Defender Firewall rules were configured and tested. Telnet was blocked from remote hosts, access to a development port was restricted to localhost, and dropped packets were logged for monitoring. Together the tasks show the two sides of security: finding weaknesses in an application, and reducing exposure at the network layer.

Scripts, raw scan reports (HTML/JSON) and screenshots are included in the submission folder and on GitHub: https://github.com/ahsanc413-rgb/Assignment4-InfoSec